

```
#whoami
nobody
```

To get the PID namespace, use the following command:

```
#unshare --pid --fork /bin/bash
```

Inside this namespace, you can see all the processes but cannot kill any.

```
#ps -aux |grep firefox
root      1110  42.6  11.0  1209424 436756  tty1    Sl   23:36
0:15 .firefox1/.firefox
root      1208   0.0   0.0   12660   1648  pts/2    S+   23:37
0:00 grep firefox
#kill 1110
bash: kill: (1110) - No such process
```

To get a whole different degree of process tree isolation you need to mount another proc for the namespace, as follows:

```
unshare --pid --fork --mount-proc /bin/bash
```

In this way, you can use unshare to create a single namespace. More about it can be found out on the man page of unshare.

 **Note:** A namespace created by using unshare can also be combined to create a single shell which uses different namespaces. For example:

```
#unshare --pid --fork --user /bin/bash
```

This will create an isolated environment using the PID and user namespaces.

Firejail

Firejail is an SUID sandbox program that is used to isolate programs for testing or security purposes. It is written in C and can be configured to use most of the namespaces. To start a service in firejail, use the following command:

```
#firejail firefox
```

It will start Firefox in a sandbox with the root file system mounted as read only. To start Firefox with only `~/Downloads` and `~/mozilla` mounted to write, use the following command:

```
#firejail --whitelist=~/mozilla --whitelist=~/Downloads firefox
```

Firejail, by default, uses the user namespace and mounts empty temporary file systems (*tmpfs*) on top of the user home

directory in private mode. To start a program in private mode, use the command given below:

```
#firejail --private firefox
```

To start firejail in a new network stack, use the following command:

```
#firejail --net=eth0 --whitelist=~/mozilla --whitelist=~/
Download firefox
```

To assign an IP address to the sandbox, use the following command:

```
#firejail --net=eth0 --ip=192.168.1.155 firefox
```



Note: To sandbox all programs running by a user, you can change the default shell of that user to `/usr/bin/firejail`.

```
#chsh -shell /usr/bin/firejail
```

Containers

When learning about virtualisation technologies, what attracted me most were containers because of their easy deployment. Containers (also known as lightweight virtualisation) are tools for isolation, which use namespaces for the purpose. They are a better sandboxing utility, because they generally use more than one namespace and are more focused on creating a whole virtual system instance rather than isolating a single process.

Containers are not a new technology. They have been in UNIX and Linux for decades but due to their increasing use in SaaS and PaaS, they have become a hot topic since they provide the most secure environment to deliver and use these services. They are called lightweight virtualisation because they provide process level isolation, which means they depend on the Linux kernel. Hence, only those instances can be created which use the same base kernel. There are lots of containers available for Linux that have gained popularity over the past few years.

Systemd-nspawn

This is a utility available by default with systemd, which creates separate containers for isolation. It uses mount and PID namespaces by default but another namespace can also be configured. To create a container or isolated shell, you need to download a basic distribution which we have done already, using `debootstrap`. To get inside this container, use the code below:

```
#systemd-nspawn -D my_deb
```

This container is stronger than *chroot* because it not only has a different mount point but also a separate process tree (check it by `ps -aux`). But still, the hostname and IP interfaces